

BACKEND DISCUSSION

Firestore already stores the chat. The extra load is sending that same chat again on every API call.

Correct summary (read this first)

- **What is already true:** The app saves messages in Cloud Firestore. That copy can be the source of truth.
- **What is wasteful today:** On every new message the Flutter app also sends the full `systemInstruction` and the full history to the backend (`POST /v1/chat`). That is a second copy of data Firestore already has.
- **What this proposal does:** Flutter sends only the new `userMessage` (and a `conversationId`). The backend reads prompt + history from Firestore, then calls `Gemini generateContent` the same way as the Dart file.
- **What this proposal does not do:** It does not summarise old messages. It does not drop turns. Gemini still receives the full stored history, in order. It does not remove the Gemini call. One new user message still means one `generateContent` call.

APIs in this note

Name	Caller	Role
Cloud Firestore Write	Flutter (already)	Saves each successful user turn and assistant turn.
Cloud Firestore Read Firestore Admin SDK	Backend (proposed)	Loads the saved system prompt and ordered messages for that conversation.
POST /v1/chat (current backend proxy)	Flutter → Node	Today: large body (prompt + full history + new text). This is the duplicate upload.
POST /v1/conversations (proposed, once)	Flutter → Node	Creates the conversation and stores the system prompt in Firestore. Returns <code>conversationId</code> .
POST /v1/conversations/{id}/messages (proposed, every send)	Flutter → Node	Small body: only the new <code>userMessage</code> . Backend reads Firestore, then calls Gemini.
POST /v1beta/models/{model}:generateContent Gemini API	Backend → Google	Unchanged contract: history first, current user last; temperature 0.7; 512 tokens; 30s timeout.

Where the load is

Place	Load?	Why
Flutter → POST /v1/chat	Yes — duplicate	The phone re-uploads prompt + all old turns every send. Firestore already has them.
Node JSON parse / RAM	Low	Building the Gemini JSON is cheap. This is not the main cost.
Firestore writes	Already paid	The app already writes turns. This proposal does not add a new write pattern.
Firestore reads (new)	New Firebase cost	Backend reads the conversation and its messages instead of trusting the phone body.
Gemini <code>generateContent</code>	Main cost + latency	Still one call per new message. Token size stays the same because history is not shortened.

Step 1 — What Flutter sends today (every message)

API: POST /v1/chat → **backend** → **Gemini** generateContent

The user typed only the new line. The request still includes the full prompt and every old turn. Those old turns are already documents in Firestore.

```
{
  "systemInstruction": "You are a friendly language-learning conversation partner... Topic:
  Restaurant. Language: Spanish.",
  "userMessage": "Hola, una mesa para dos por favor",
  "history": [
    { "text": "Bienvenido. ¿Cuántas personas?", "isUser": false },
    { "text": "Somos dos", "isUser": true },
    { "text": "¿Interior o terraza?", "isUser": false }
  ]
}
```

Step 2 — Start a conversation once

API: POST /v1/conversations

Save the system prompt in Firebase at the start. Do not send that long prompt again on later messages.

```
{
  "mode": "role_play",
  "systemInstruction": "<exact string from RolePlayGeminiPrompt.build(>\"",
  "ownerId": "firebaseAuthUid_abc"
}
```

Response 201:

```
{ "conversationId": "chat_123" }
```

Firestore: conversations/chat_123 stores systemInstruction, ownerId, mode.

Step 3 — Every later message (small body)

API: POST /v1/conversations/chat_123/messages

Flutter only sends the new text. The backend rebuilds the Gemini request from Firebase. History is the full stored list, in the same order as today (old turns first, new user message last).

```
{ "userMessage": "Hola, una mesa para dos por favor" }
```

Backend then:

1. Firestore READ conversations/chat_123
2. Firestore READ conversations/chat_123/messages (full ordered history, not a summary)
3. Gemini generateContent (same fields as GeminiService.dart)
4. Firestore WRITE user + model turns if Gemini succeeds
5. Response { "text": "¡Claro! ¿Fumador o no fumador?" }

If Gemini fails or times out: { "text": null } and do not write the failed turn.

What stays the same

Role-play and free-chat prompts from the submitted Dart files; difficulty keys; user/model roles; temperature 0.7; maxOutputTokens 512; 30-second timeout; four safety categories at BLOCK_MEDIUM_AND_ABOVE; first candidate first text part, trimmed; null on failure. Groq, streaming, dashboards, and queues are out of scope.

This note is not a request to summarise or truncate chat history. It only moves the existing history from the HTTP body to a Firestore read.